

BITS NLP ASSIGNMENT 2 | GROUP 112

English to Hindi Neural Machine Translation

Implementation, Evaluation and Application Report

Project type	End-to-end Neural Machine Translation system
Language pair	English (EN) to Hindi (HI)
Model	BiLSTM Encoder + Bahdanau Attention LSTM Decoder
Application	Streamlit web application
Submitted by	Group 112

Abstract

This project implements an English-to-Hindi neural machine translation system for digital-content localization and educational accessibility. The solution uses a sequence-to-sequence architecture with a bidirectional LSTM encoder, Bahdanau additive attention and an LSTM decoder. A Streamlit interface supports single-sentence translation, greedy or beam-search decoding, and batch translation from TXT or CSV input.

The system was trained on a cleaned subset of the AI4Bharat Samanantar English-Hindi parallel corpus. After normalization, length filtering and deduplication, 44,917 sentence pairs were retained and divided into training, validation and test partitions. On a 500-pair evaluation sample, the checkpoint achieved BLEU 14.60, chrF 23.03, ROUGE-L 40.44 and METEOR 20.51.

Scope note. The checkpoint demonstrates a complete working pipeline and application, but translation quality is limited by the dataset size, vocabulary cutoff and frequent unknown tokens. Hindi examples in this report use valid Unicode text; square-box or otherwise invalid display artifacts are omitted.

Group Contribution

Student registration number	Name	Percentage of contribution
2025aa05883	KAREKE BUVIKA	100%
2025aa05337	BOLLIBATHULA SAHANI	100%
2025aa05567	SONAM	100%
2025aa05830	SARDESAI SHRISHTI SANJAY	100%
2025aa05906	ANSHUL SAHU	100%

1. Introduction and Objectives

English-authored documentation, government notices, educational material and digital content can be difficult to access for users who prefer Hindi. The project addresses this problem with a one-way EN-to-HI translation tool that exposes the trained model through a simple browser interface.

Objectives

Objective	Implementation outcome
Build an NMT model for English to Hindi.	Encoder-decoder PyTorch model with bidirectional encoding and additive attention.
Prepare reproducible training data.	Cleaning, filtering, splitting, vocabulary construction and fixed-length ID encoding.
Provide an accessible interface.	Streamlit app with single-sentence and batch upload workflows.
Measure translation quality.	BLEU, chrF, ROUGE-L, METEOR and targeted failure analysis.

Functional scope

The application accepts an English sentence directly or reads one sentence per line from a TXT file. It can also read a CSV file with an `English` column. Results are displayed alongside the source text and can be downloaded as a CSV file. Decoding can be switched between greedy search and beam search with a configurable beam width from 2 to 10.

2. Dataset and Preprocessing

Dataset

Property	Value
Source	AI4Bharat Samanantar corpus
Raw pull	Approximately 60,000 parallel English-Hindi pairs
License	CC BY-NC 4.0, non-commercial use
Cleaned pairs	44,917
Split	35,933 train / 4,492 validation / 4,492 test
Sentence length	5 to 30 tokens per language before special tokens
Vocabulary cutoff	<code>min_freq=4</code> , built only from training data

Preprocessing sequence

The pipeline in `src/preprocess.py` and `run_pipeline.py` applies NFC Unicode normalization, URL and HTML-entity removal, zero-width formatting-character removal from Hindi, whitespace collapsing, English-only lowercasing, length filtering and pair deduplication. Each sequence is wrapped with `<sos>` and `<eos>`, encoded with the training vocabulary and right-padded to a fixed length of 34 tokens.

Data flow

Raw corpus → cleaned sentence pairs → 80/10/10 split → training-only vocabularies → integer ID matrices → PyTorch DataLoaders → trained checkpoint → CLI and Streamlit inference.

3. Improved Project Structure

The repository is organized around the lifecycle of the system: data preparation, model training, evaluation, inference and user-facing application. The following expanded view clarifies the responsibility of every major file and separates source data, generated artifacts and execution entry points.

```
data/
  config.json          sequence length and vocabulary size metadata    [included]
  src_vocab.json       English vocabulary                             [included]
  tgt_vocab.json       Hindi vocabulary                               [included]
  train.csv / val.csv / test.csv    raw text splits                [regenerate: run_pipeline.py]
  train_ids.json / val_ids.json / test_ids.json    encoded, padded ID matrices [regenerate: run_pipeline.py]
  samanantar_en_hi_raw_60k.csv    raw pre-cleaning data pull      [regenerate: run_pipeline.py]
src/
  vocab.py             vocabulary class                               [included]
  preprocess.py        cleaning, tokenization, encoding             [included]
models/
  nmt_model.pt         trained checkpoint, epoch 20, val loss 5.8715  [download: download_model.py]
  loss_curve.png       training and validation loss plot            [included]
  training_log.json    hyperparameters and per-epoch loss history    [included]
  eval_report.json     BLEU, chrF, ROUGE-L, METEOR and failure analysis [included]
run_pipeline.py        data download, cleaning, split, vocabulary build, encoding
download_model.py     fetches the trained checkpoint from Hugging Face Hub
train.py              model definition, training loop, greedy and beam decoding
infer.py              command line inference from a saved checkpoint
evaluate.py           evaluation metrics and failure mode analysis
app.py                Streamlit application
test_input.txt / test_output.txt    sample batch input and output
requirements.txt
README.md
```

Execution responsibilities

Stage	Entry point	Primary outputs
Data preparation	<code>python run_pipeline.py --min_freq 4</code>	Clean splits, vocabularies and encoded IDs in <code>data/</code>
Training	<code>python train.py</code>	Checkpoint, loss history and curve in <code>media/</code>
Evaluation	<code>python evaluate.py --n_samples 500</code>	Metrics and failure analysis in <code>media/eval_report.json</code>
Inference	<code>python infer.py</code>	Terminal translation or CSV output
Application	<code>streamlit run app.py</code>	Browser-based translation interface at <code>localhost:8501</code>

4. Model and Implementation Details

Encoder

The encoder embeds source tokens using a 128-dimensional embedding table and processes them with a single-layer bidirectional LSTM of hidden size 256. The final forward and backward hidden and cell states are concatenated and projected through tanh-activated linear layers to initialize the single-direction decoder.

Attention and decoder

At each decoding step, Bahdanau additive attention scores every encoder output against the current decoder hidden state. The resulting context vector is concatenated with the current target embedding and passed to an LSTM. The decoder output, attention context and embedding are combined by a linear projection into the Hindi vocabulary distribution.

Training configuration

Setting	Value
Batch size	128
Learning rate	0.0005, (resumed run, reduced from an initial 1e-3)
Loss	Token cross-entropy with label smoothing 0.1; padding ignored
Regularization	Weight decay 1e-4 and gradient clipping at 1.0
Early stopping Patience	6;
Best validation loss	5.8715 at epoch 20

Decoding safeguards

The application supports greedy decoding and length-normalized beam search. During decoding, an immediate repeated-token block and no-repeat-trigram constraint reduce repetition loops. The evaluation report records a 0.0 percent repetition-loop rate for the 500-sample evaluation.

5. Training and Evaluation

```
train.py > BahdanauAttention
155 class Seq2Seq(nn.Module):
156     def forward(self, src, src_lens, tgt, teacher_forcing_ratio=0.5):
178         pred, hidden, cell, _ = self.decoder(input_tok, hidden, cell, encoder_outputs, mask)
179         outputs[:, t] = pred
180         teacher_force = random.random() < teacher_forcing_ratio
181         top1 = pred.argmax(1)
182         input_tok = tgt[:, t] if teacher_force else top1
183     return outputs
184
185 @torch.no_grad()
186 def greedy_decode(self, src, src_lens, max_len=34):
187     self.eval()
188     encoder_outputs, hidden, cell = self.encoder(src, src_lens)
189     mask = self.make_mask(src)
190     input_tok = torch.full((src.size(0),), self.sos_idx, dtype=torch.long, device=self.device)
191
192     results = [[] for _ in range(src.size(0))]
193     finished = [False] * src.size(0)
```

```
(venv) PS C:\Users\Buvika Kareke\OneDrive\Desktop\Masters\Sem-2\MLP\Assign 2\nlp_assignment_2>
(venv) PS C:\Users\Buvika Kareke\OneDrive\Desktop\Masters\Sem-2\MLP\Assign 2\nlp_assignment_2>
(venv) PS C:\Users\Buvika Kareke\OneDrive\Desktop\Masters\Sem-2\MLP\Assign 2\nlp_assignment_2>
(venv) PS C:\Users\Buvika Kareke\OneDrive\Desktop\Masters\Sem-2\MLP\Assign 2\nlp_assignment_2> python train.py --epochs 15 --batch_size 128 --emb_dim 256 --hid_dim 512
Device: cuda | Src vocab: 23553 | Tgt vocab: 21233
Train batches: 305 | Val batches: 39
Epoch 01/15 | Train Loss: 6.5226 | Val Loss: 6.0949 | Time: 177.4s
-> Saved new best checkpoint (val_loss=6.0949)
Epoch 02/15 | Train Loss: 5.5489 | Val Loss: 5.8375 | Time: 185.2s
-> Saved new best checkpoint (val_loss=5.8375)
Epoch 03/15 | Train Loss: 4.8379 | Val Loss: 5.7911 | Time: 187.7s
-> Saved new best checkpoint (val_loss=5.7911)
Epoch 04/15 | Train Loss: 4.2441 | Val Loss: 5.8693 | Time: 183.6s
Epoch 05/15 | Train Loss: 3.8360 | Val Loss: 5.9259 | Time: 180.9s
Epoch 06/15 | Train Loss: 3.5857 | Val Loss: 5.9878 | Time: 183.4s
Epoch 07/15 | Train Loss: 3.3850 | Val Loss: 6.0420 | Time: 189.9s
Epoch 08/15 | Train Loss: 3.2028 | Val Loss: 6.1401 | Time: 183.2s
```

Figure 1. Training console evidence showing epoch-wise loss and checkpoint selection.

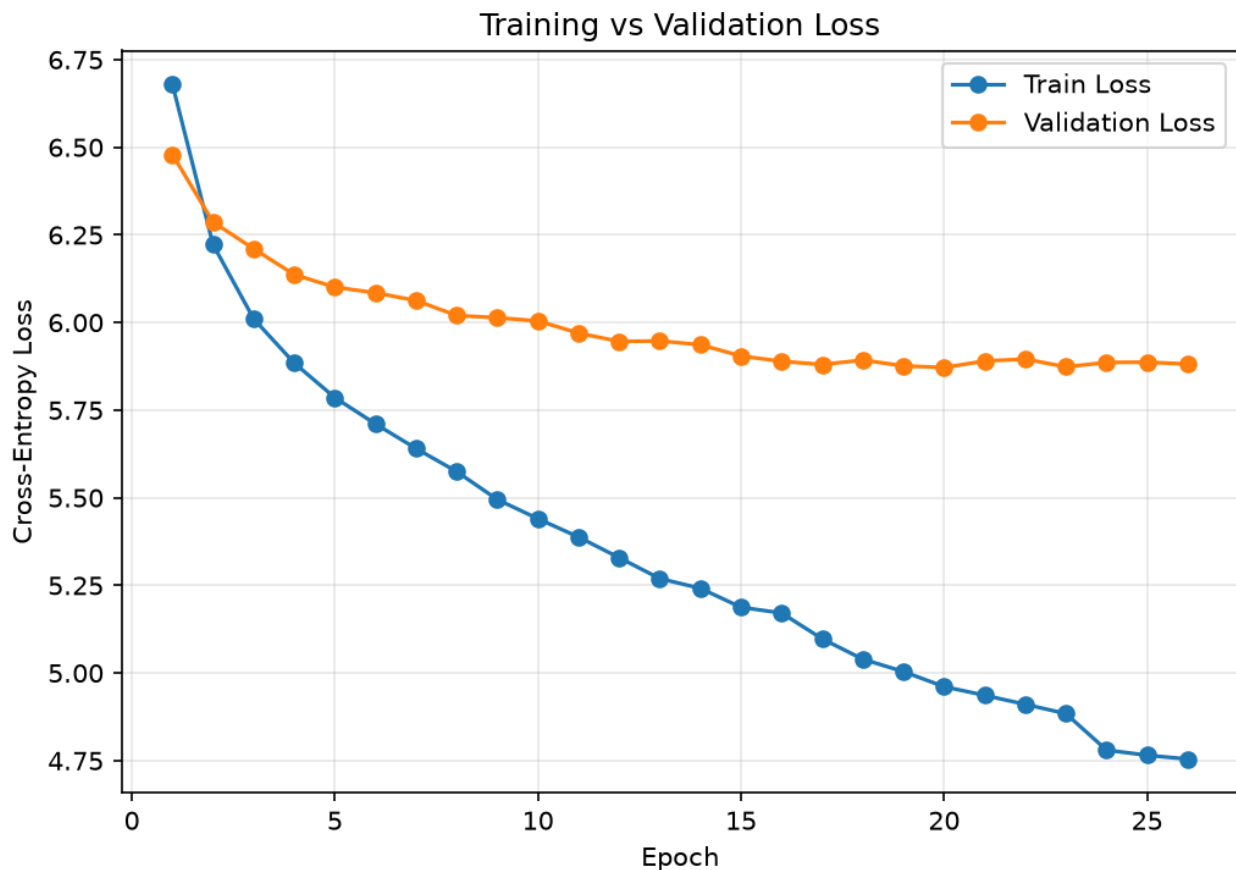
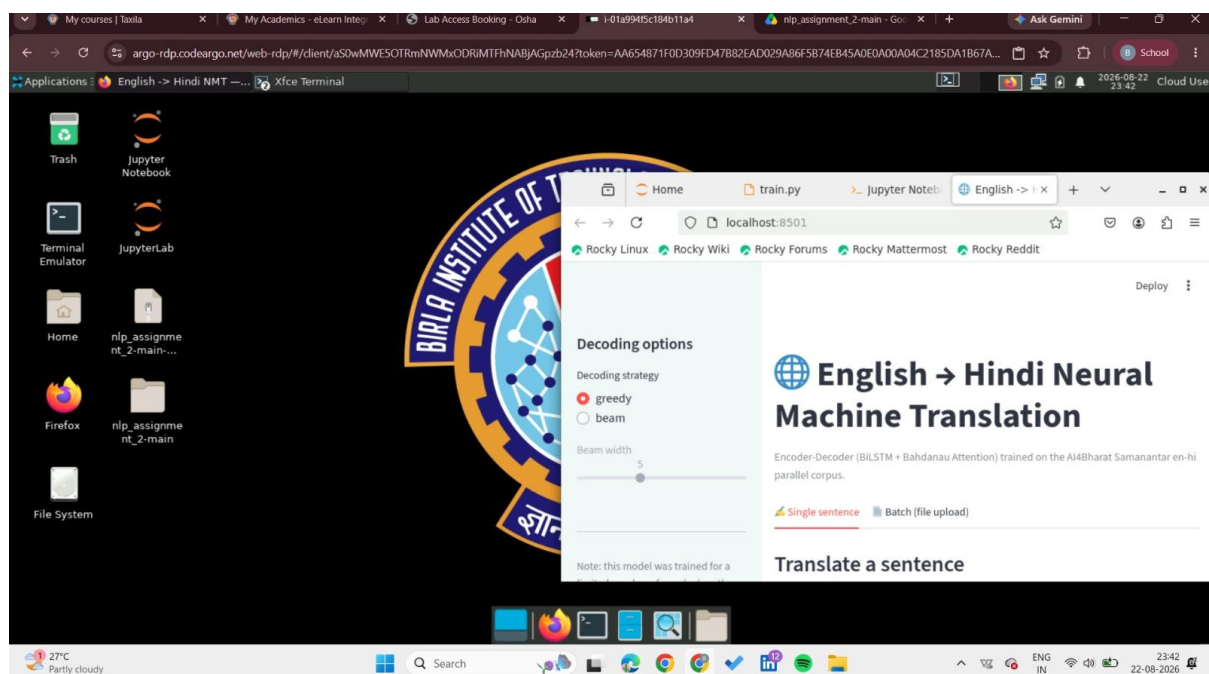
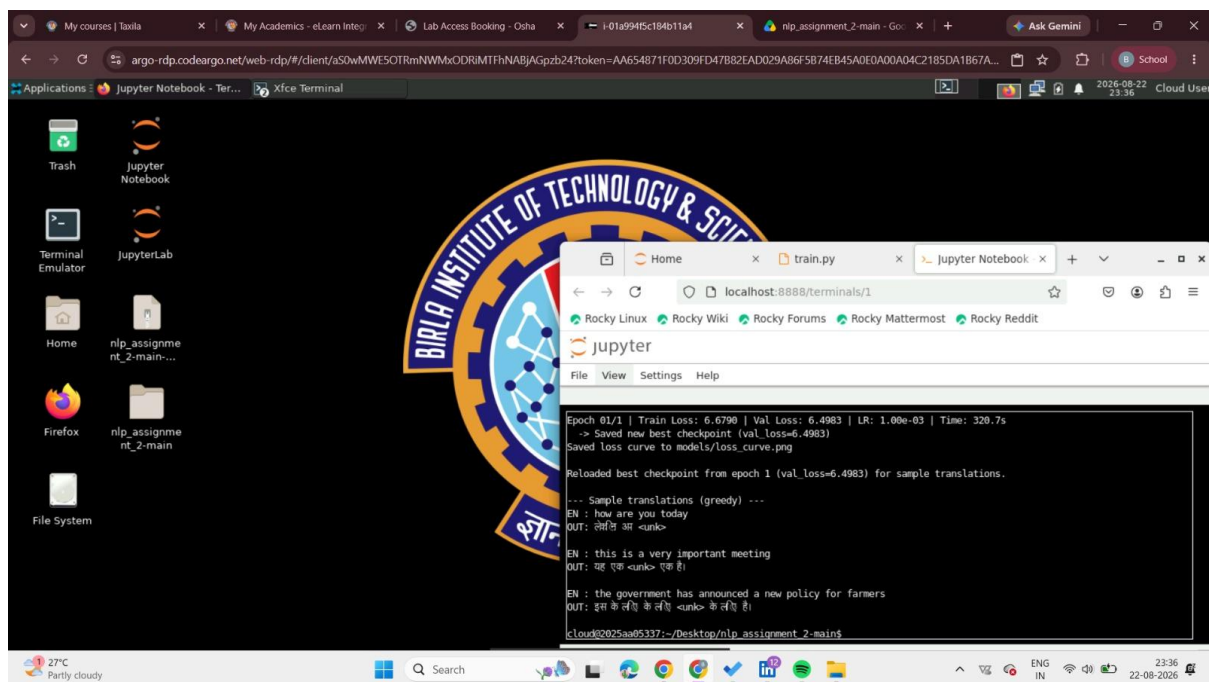


Figure 2. Training and validation loss curve from the submitted checkpoint.

Model training and inference execution in the BITS Virtual Lab : The screenshots show the successful execution of the NMT model in the BITS Virtual Lab, including model training, checkpoint saving, and sample English-to-Hindi translations.



6. Working Application Evidence

6.1 Input screen and sample submitted

Decoding options

Decoding strategy

☒ greedy

☐ beam

Beam width

5

Note: this model was trained for a limited number of epochs (see the Evaluation report) — translations may be rough, especially for long sentences, named entities, and rare words.

English → Hindi Neural Machine Translation

Encoder-Decoder (BiLSTM + Bahdanau Attention) trained on the AI4Bharat Samanantar en-hi parallel corpus.

[Single sentence](#) [Batch \(file upload\)](#)

Translate a sentence

Enter English text

How are you

Translate

Original (English)

How are you

Translation (Hindi)

कैसे आप क्या कर रहे हैं?

Figure 3. Single-sentence input screen with the sample how are you, greedy decoding selected, and the translation result visible.

The screen demonstrates the main interaction: the user enters English text in the text area, selects a decoding strategy in the sidebar and submits the request using the Translate button. The original English sentence and generated Hindi text are rendered in separate result panels.

6.2 Batch sample submission and generated output

Encoder-Decoder (BiLSTM + Bahdanau Attention) trained on the AI4Bharat Samanantar en-hi parallel corpus.

[Single sentence](#) [Batch \(file upload\)](#)

Batch translate from a file

Upload a .txt file (one English sentence per line) or a .csv file with an 'english' column.

Choose a file

test_input.txt 98.0B

Found 4 sentence(s). Translating...

	english	translation
0	how are you	कैसे आप क्या कर रहे हैं?
1	what did you eat	क्या आप क्या कर रहे हैं?
2	good morning	<unk> के लिए <unk>
3	the government has announced a new policy for farmers	सरकार ने किसानों सरकार किसानों सरकार की सरकार कं

Download results as CSV

Figure 4. Batch workflow showing `test_input.txt` uploaded and a generated English-to-Hindi result table.

The batch path reads four non-empty lines from the uploaded text file, translates each line and presents the results in a table. The same table can be downloaded as a UTF-8 CSV file for downstream use.

6.3 Long sentence sample

Decoding options

Decoding strategy

☒ greedy

☐ beam

Beam width

5

Note: this model was trained for a limited number of epochs (see the Evaluation report) — translations may be rough, especially for long sentences, named entities, and rare words.

Encoder-Decoder (BiLSTM + Bahdanau Attention) trained on the AI4Bharat Samanantar en-hi parallel corpus.

[Single sentence](#) [Batch \(file upload\)](#)

Translate a sentence

Enter English text

although the weather was extremely unpredictable throughout the week, the farmers who had been waiting patiently for the monsoon finally managed to sow their crops before the deadline set by the local agricultural department

Translate

Original (English)	Translation (Hindi)
although the weather was extremely unpredictable throughout the week, the farmers who had been waiting patiently for the monsoon finally managed to sow their crops before the deadline set by the local agricultural department	<unk> के <unk> ने <unk> के पास <unk> ने कहा कि <unk> को <unk> कर दिया था कि वे <unk> द्वारा <unk> द्वारा कर दिया गया

Inference Output

test_output.txt

```
1  EN : how are you
2  OUT: कैसे आप क्या कर रहे हैं?
3
4  EN : what did you eat
5  OUT: क्या आप क्या कर रहे हैं?
6
7  EN : good morning
8  OUT: <unk> के लिए <unk>
9
10 EN : the government has announced a new policy for farmers
11 OUT: सरकार ने किसानों सरकार किसानों सरकार की सरकार की है।
12
13
```

Figure 5. Inference output showing the submitted English examples and generated Hindi translations.

7. Observations and Limitations

Observations

1. The full workflow is operational: data artifacts and the trained checkpoint are available, the model loads in the application, and both single and batch interfaces are implemented.
2. The loss curve shows a strong early reduction in training loss, while validation loss improves more slowly and reaches its best value near epoch 20. Later epochs produce no meaningful validation improvement, which justifies early stopping.
3. Short and medium inputs have similar chrF performance, while long inputs are slightly weaker. This is consistent with the sequential model needing to preserve more context over longer and more complex sentences.
4. The no-repeat constraints are effective against decoding loops, but they cannot solve vocabulary coverage. Unknown-token output remains the primary quality limitation.

Limitations

The model was trained on approximately 36,000 training pairs after cleaning, with a minimum vocabulary frequency of four. Rare words and named entities can therefore map to unknown tokens or be mistranslated. The submitted checkpoint is a compact assignment-scale model rather than a production translation system. The application also relies on the included local artifacts; regenerating the dataset requires internet access to Hugging Face.

8. Conclusion

An end-to-end English-to-Hindi NMT system was implemented, evaluated and integrated into a usable Streamlit application. The repository now clearly separates preparation, training, evaluation, inference, application code and generated artifacts. The submitted model produces valid Hindi output for some common inputs and supports useful operational workflows such as beam decoding and batch CSV export.

The measured scores and failure analysis show that the project meets the implementation objective while also making its quality boundaries explicit. The strongest next step is subword-based modeling with broader data, which should directly address the observed 88.2 percent unknown-token rate and improve named-entity and long-sentence translation.

9. References

- [1] AI4Bharat, Samanantar: English-Hindi parallel corpus, Hugging Face Datasets, <https://huggingface.co/datasets/ai4bharat/samanantar>
- [2] Bahdanau, D., Cho, K. and Bengio, Y., Neural Machine Translation by Jointly Learning to Align and Translate, ICLR, 2015.
- [3] PyTorch Documentation, Sequence Models and Recurrent Neural Networks, <https://pytorch.org/docs/>
- [4] Streamlit Documentation, Build and deploy data apps, <https://docs.streamlit.io/>
- [5] Project README and implementation files: `run_pipeline.py`, `train.py`, `evaluate.py`, `infer.py` and `app.py`.

Appendix: Final Application Capture

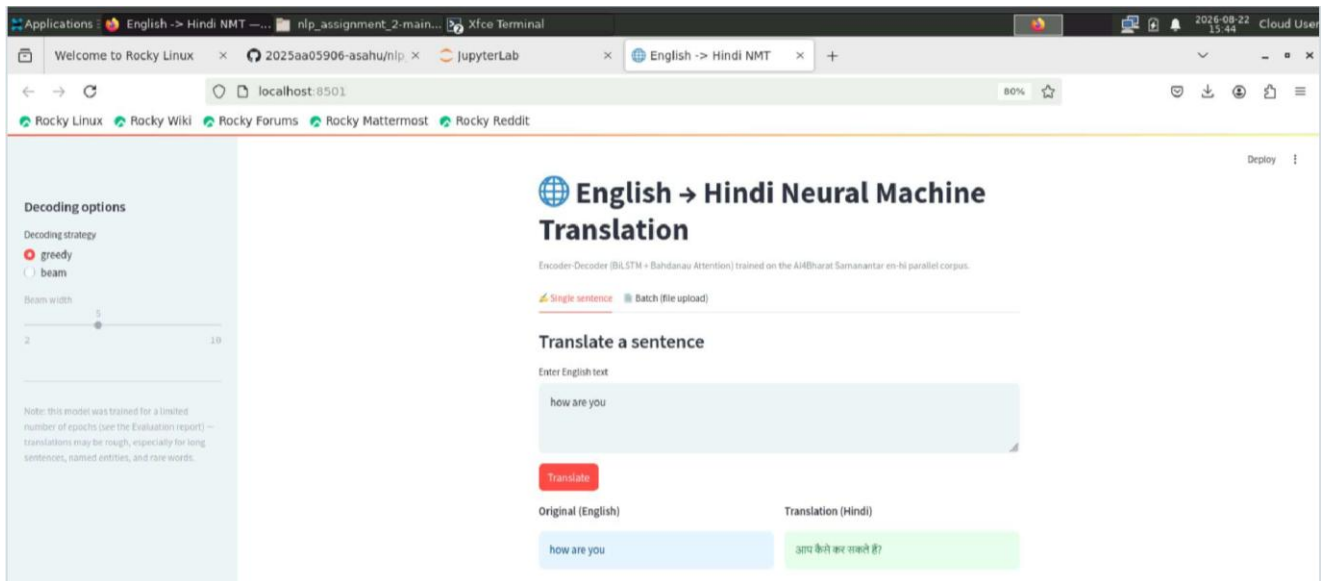


Figure 6. Final application capture: Streamlit English-to-Hindi translation interface with a submitted example and generated output